



Implementation of the language server protocol and a
Visual Studio Code extension for C/ACSL programming
language features.

August 17, 2024

Internship report

Djamila Mohamed
2023–2024

Supervisor:
Adel Djoudi, Formal Methods Expert

Contents

1 Introduction

1.1 Thales DIS

Thales Digital Identity and Security is the global business unit of Thales which operates in the field of secure software, providing biometric and encryption solutions. Within DIS is the Identity and Biometric Solutions business line which is in charge of digital identity (e.g.: cards and passports).

The IBS BL is divided in three subunits: ID Documents Solutions (IDDS), Digital ID & Verification Solutions and Public Security. I am assigned to the IDDS subunit as an intern software developer, under the supervision of Adel Djoudi, formal methods expert at Thales DIS.

1.2 Context

1.2.1 Frama-C

In IDDS, developers use program analyzers such as Frama-C¹, a C source file analyzer developed by CEA and Inria².

Frama-C is a sophisticated toolkit for analyzing source code written in C. It combines various static and dynamic analysis methods into a single system, allowing different tools included in the suite to build on each other's results for enhanced accuracy.

A key feature of Frama-C is its support for the ANSI/ISO C Specification Language (ACSL), a formal language designed for specifying the behavior of C programs. ACSL facilitates the precise expression of functional properties through function contracts. It enables both human-readable and machine-processable specifications, ensuring a detailed understanding of function behavior.

Frama-C's static analysis is oriented towards ensuring code correctness. Frama-C guarantees the identification of all possible runtime errors and validates adherence to specified functional requirements.

The collaborative framework of Frama-C enhances its analytical power, integrating various tools and allowing them to contribute to a comprehensive examination of code. Its modular and extensible design supports a wide range of analysis techniques, making it a good choice for ensuring code reliability and compliance to detailed functional specifications.

1.2.2 Problem

Frama-C currently lacks a dedicated integrated development environment (IDE) for editing and browsing C code and ACSL annotations, leading to significant inefficiencies in the development process. Writing ACSL specifications often requires manually searching through the Frama-C libraries to find the relevant functions and their definitions. This approach is not only cumbersome but also highly inefficient.

For those familiar with Frama-C, locating function definitions involves using the `frama-c -print-share-path` bash command to identify the directory where Frama-C libraries are stored. Following this, the `grep` utility can be used to search for specific predicate definitions within these files. For example, the command `grep -r 'predicate_name' /path/to/frama-c/libraries` is employed to recursively search for occurrences of 'predicate_name' within the directory. While this method can yield results, it requires manual navigation and is prone to inefficiencies, especially in large codebases with extensive libraries. Below is an example with the predicate 'valid_read_string'.

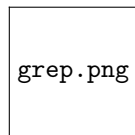


Figure 1: Example of searching predicate definition with grep

Searching through potentially large directories and files using `grep` is time-consuming, especially when dealing with complex libraries with numerous functions and predicates. This method relies heavily on manual effort and familiarity with command-line tools, which can be error-prone and inconvenient.

The inefficiencies of the current method emphasize the necessity for a more streamlined solution. A Language Server Protocol (LSP) tailored for ACSL code would offer significant improvements by providing features such as real-time code browsing, automatic function and predicate lookup, contextual information, and integration with the editing environment. Such a tool would greatly enhance productivity by automating these tasks and reducing the reliance on cumbersome manual search processes.

¹<https://frama-c.com/html/overview.html>

²two prominent research institutions with overlapping goals in scientific advancement and technology innovation

Additionally, those unaccustomed to Frama-C and ACSL annotations can benefit from this solution for getting started and have a better understanding of the specification language.

The goal of this project is to develop a software solution to offer language programming features for C/ACSL in IDEs including code navigation, completion suggestions, and interaction with Frama-C code analysis, to enhance validation and facilitate certification at EAL6/EAL7 levels of the Common Criteria.

1.3 Work organization

2 Background

2.1 The Language Server Protocol

2.1.1 Introduction

Developers working on large scale projects often need a code editing environment with the adequate tooling for the languages they use in their workspace. Multiple solutions such as Eclipse or IntelliJ IDEA exist for Java projects, as well as other popular programming languages. But the complexity of developing one tool for one language and one specific editor is tedious as it forces developers to implement the same language programming features for every code editor with each programming language. To overcome these, a solution was developed: the Language Server Protocol (LSP)³.

The language Server Protocol (LSP) is built on top of the [JSON-RPC protocol](#), a remote procedure call protocol that uses JSON as data format. It permits a communication between a client and a server.

TODO : parler des methodes et des types de requetes

TODO : mettre les schemas du site du lsp, tous les clients doivent respecter le LSP (le format JSON)

2.1.2 LSP Capabilities

The LSP defines sets of features named ‘capabilities’. LSP capabilities are boolean values transported in a JSON request that each party declares when the extension is launched. These declarations are needed so that both agree on which features each can provide. Only the features supported by both parties can be used in the editor. Capabilities can be registered in two ways: statically and dynamically. Static registration is done through the initialize request, so it is done once and for all. Dynamic registration allows language servers to add or remove support for various capabilities at runtime without a reinitialization of the server.

Below is a sequence diagram of the first requests exchanged between the client and the server, the user can start using the extension once the server has received the ‘initialized’ notification by the the client (not to be confused with the ‘initialize’ request)

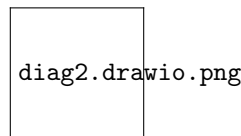


Figure 2: Sequence diagram of the ‘initialize’ request

2.1.3 Features

TODO : classification of features and example of features and example of corresponding json for each feature, ref to the appendix for full listing of lsp features

2.2 Frama-C plugin development

describe working example, comment on crée on crée un plugin, dune (librairies, entry point)

describe the kernel api, wp api and modules that are relevant to implement the LSP

parler de l’invariant : un seul ast par projet

Frama-C is an extensible platform for the static analysis of C programs. Its architecture allows for the development of custom plugins, which can leverage the existing infrastructure and libraries provided by Frama-C to perform specialized analyses. This section provides an overview of the process and tools available for developing a Frama-C plugin, focusing on the key libraries, modules, and APIs that support plugin development.

2.2.1 Frama-C Architecture Overview

The Frama-C framework is composed of several core components that facilitate the development of plugins:

³<https://microsoft.github.io/language-server-protocol/overviews/lsp/overview/>

Kernel Library

The Kernel is the backbone of Frama-C, responsible for managing the project lifecycle, abstract syntax trees (AST), and the overall data flow within the platform. It provides essential services such as parsing C source files, type checking, and maintaining the internal representation of the program being analyzed.

Control Flow Graph (CFG)

The CFG plug-in is central to performing control flow analysis in Frama-C. It allows the construction and manipulation of control flow graphs, which represent all possible paths of execution in a program. Plugins that need to analyze or transform the control flow of a program rely heavily on this module.

Deductive Verification (WP)

The WP plugin is used to generate verification conditions using the weakest precondition calculus. These conditions are then checked to ensure that the program adheres to its formal specification, typically expressed in ACSL (ANSI/ISO C Specification Language). This module is crucial for plugins aimed at formal verification or proof-based analysis.

Metrics

This module computes various code metrics, such as cyclomatic complexity and lines of code. These metrics are often used to assess the quality and maintainability of the code, making this module useful for plugins focused on code quality analysis.

2.2.2 Steps in Developing a Plugin

Developing a plugin for Frama-C involves several methodical steps, each crucial for ensuring that the plugin integrates seamlessly with the existing Frama-C infrastructure:

Environment Setup: Plugin development in Frama-C requires proficiency in OCaml, as both Frama-C and its plugins are implemented in this language. Developers must set up an OCaml development environment and obtain the Frama-C source code along with its dependencies.

Plugin Initialization: The development of a Frama-C plugin begins with defining the plugin module in OCaml. This involves registering the plugin with the Frama-C Kernel, specifying metadata such as the plugin's name, version, and a brief description.

Interfacing with the Kernel: Once the plugin is initialized, it must interface with the Frama-C Kernel to access the AST, CFG, or other program representations. The Kernel API provides various functions to manipulate these representations, allowing the plugin to register callbacks or hooks during specific analysis phases.

Utilizing Core Modules: Depending on the analysis goals, the plugin may need to interact with specific Frama-C modules like CFG or WP. For instance, a plugin designed to analyze the control flow of a program will use the CFG module to traverse and manipulate the CFG.

Testing and Debugging: During the development process, it is essential to test and debug the plugin thoroughly. Frama-C offers logging and debugging facilities that allow developers to track the plugin's execution and identify potential issues.

Command-Line Integration: After implementing the core functionality, the plugin needs to be integrated with Frama-C's command-line interface (CLI). This integration enables users to invoke the plugin directly from Frama-C, using standard command-line options.

Documentation and User Support: Frama-C encourages developers to document their plugins thoroughly. This includes embedding help messages within the plugin and providing external documentation to assist users in understanding the plugin's functionality and usage.

2.2.3 Supporting Tools and Resources for Plugin Development

Frama-C offers a comprehensive set of tools and resources to assist developers in creating high-quality plugins:

API Documentation: The Frama-C API provides an online API where we can browse each plug-in's library and better understand its functions.

Plugin Guide: Frama-C provides a tutorial for developing a new plugin, serving as a starting point that includes basic setup and interaction with the Frama-C Kernel library.

Community Support: Frama-C community provides mailing lists and forums for plugin developers.

2.3 VS Code Extensions

working example (extension plaintext), comment est organisé une extension typescript

The architecture of VS Code extensions is based on a client-server model, which aligns perfectly with the Language Server Protocol. VS Code provides an API with TypeScript libraries for developing different types of extensions, particularly, language extensions. A language extension offers declarative (e.g. syntax highlighting, bracket autoclosing) and programmatic (e.g. hover information, jump to definition) language features.

2.3.1 ACSL extensions

Very few VS Code extensions for Frama-C exist in the marketplace⁴. The only extensions related to Frama-C are for ACSL syntax coloration made by an independent developer⁵.

The first one, ‘ACSL-lite’⁶ removes syntax coloration for C code and adds coloration for ACSL annotations, but it has no other feature. The drawback of this extension is that it removes entirely the coloration for C when we also need it.

The second one ‘ACSL: ANSI/ISO C Specification Language’⁷ has coloration for both ACSL and C. But there are a few fixes to make.

3 Design Choices

3.1 Server

The server was implemented as a Frama-C plug-in to fully exploit the advanced parsing and analysis capabilities inherent in the Frama-C framework. Frama-C is a powerful platform for source code analysis of C programs, offering a wide range of tools and libraries tailored for static analysis. By developing the server as a plug-in within this ecosystem, we can directly leverage these pre-existing tools.

The server itself is an OCaml application that facilitates communication between the Frama-C plug-in and external clients. This design choice allows us to maintain a modular architecture where the core analysis logic remains within the Frama-C environment, ensuring that it benefits from continuous updates and improvements made to the Frama-C platform. The server acts as an intermediary, managing interactions between the Frama-C plug-in and the client, which can be an independent application requiring code analysis services.

This architecture—comprising the server executable, the Frama-C plug-in, and the client—ensures a clear separation of concerns. The server handles communication and orchestration, the plug-in performs the specialized analysis tasks, and the client interfaces with the server to request these services. This modularity not only enhances maintainability but also allows for flexibility in extending or adapting any of the components as needed.

3.2 Communication

Unix sockets were selected as the communication mechanism among the three components—server, Frama-C plug-in, and client—due to their efficiency and simplicity in local inter-process communication (IPC). Unlike other IPC methods such as pipes or shared memory, Unix sockets provide a versatile and robust means of exchanging data between processes running on the same machine.

The decision to use Unix sockets over other alternatives like TCP/IP sockets was driven by the need for low-latency and secure communication in a local environment. Since the processes are all located on the same host, Unix sockets offer a significant performance advantage by avoiding the costs associated with network protocol layers, which are unnecessary in this context.

Furthermore, the OCaml Unix module, which provides native support for Unix sockets, allows ideal integration with the existing OCaml-based infrastructure of the server. This compatibility ensures that data is transmitted efficiently. By leveraging Unix sockets, we achieve a communication mechanism that is not only fast and reliable but also well-suited to the architecture of the application.

parler du handler (1 command par feature)

algo du handler

3.3 Architecture

vscode = serveur

The first purpose of this project is to implement a language server for ACSL/C, so the project will forcibly have a directory with the server code in it. But we also want a client that will use it and we chose vs code.

⁴The Visual Studio marketplace where VS Code extension can be installed

⁵<https://github.com/MisterZurg>

⁶<https://github.com/MisterZurg/acsl-lite>

⁷<https://github.com/MisterZurg/acsl-ansi-iso-c-specification-language>

Now we have a default client, so there is a need for a directory with the source code of the client. We decided to put the client in the same project as the server so when the user installs it it comes as a bundle, instead of letting the user install them separately, since the source code of the vs code client consists of only file.

As said earlier, in theory, the server should work with any client, but we have not tested with other clients, and we do not know how the other clients handle the server process: for example, vs code creates the process and the frama-c plug-in connects to it, but do not know if other plug-ins function the same way. So all features and issues encountered during this internship, concern the combination of the Frama-C plug-in with the VS Code extension. In this report, we will refer to the plug-in directly as the server to avoid confusion.

We chose the architecture of our project based on how the LSP leads us to: we create one OCaml module per feature, (e.g.: ‘Go To Definition’ feature will be handled by a module written in file ‘definition.ml’).

3.4 A first approach on the server implementation

In the context of Frama-C, an execution instance, whether initiated via the command line or through a function call within the Frama-C library, is referred to as a session. Initially, we considered implementing the server directly as a Frama-C plug-in, whereby the main loop responsible for handling Language Server Protocol (LSP) requests would be executed within this plug-in. This approach would handle all LSP requests into a single, continuous session.

However, this design presented a significant drawback: when Frama-C encounters a syntax error during file parsing, the entire session enters an error state. Given that a session in this context would persist until the server either crashes or the client terminates it, the error state would persist indefinitely. This is problematic because even if the user corrects the error in the code, the session—and consequently the server—would remain in the error state.

Terminating the session as a response to an error is not a viable solution, as it would lead to the server’s shutdown. This, in turn, would cause the client to disconnect, leading to a crash of the extension. Consequently, the user would need to manually restart the extension, which is a cumbersome and undesirable outcome. To avoid such disruptions and maintain a seamless user experience, it is imperative that the server architecture be designed to handle errors more gracefully without necessitating a session termination.

3.5 plug-in installation

4 Development

working example (extension plaintext), comment est organisé une extension typescript

4.1 Project configurations

tableau feature/parsing local (.h ou .c) ou global (spécifié par user ou par défaut que les fichiers .c)

tableau de chaque feature et sa ligne de commande frama-c (usage de -then et -then-last) faire le lien avec les configs

listing de toutes les options, leur organisation

tableau feature/module api frama-c utilisé

tableau algo principal de chaque feature

tableau des capacités du serveur

4.2 ‘Go To Definition’ feature

The ‘Go To Definition’ feature of Visual Studio Code allows developers to easily navigate to the definition of symbols within their code. In this context, the challenges involve implementing a VS Code extension that supports this feature for two languages associated with the same file extension (‘.c’ and ‘.h’).

When a user requests to find the definition of a symbol at a specific position in the code, the server first identifies the name of the symbol located at that position. This involves parsing, analyzing the file and extracting the relevant word based on the provided line and column numbers. The code then initiates a search through various subsections of the code to find where the symbol is defined. This search involves visiting different parts of the source file’s Abstract Syntax Tree (AST) and concerns both ACSL and C code:

- ACSL Definitions: The code checks if the symbol matches any logical predicates or terms, which are specific to tools like Frama-C used for static analysis of C programs.
- C Definitions: It examines global entities, such as enums, structures, functions, and variables, to see if the symbol matches any of these.

Each of these searches uses visitor patterns, which traverse and inspect nodes in the AST, comparing each node's name to the symbol's name.

If a definition is located, the exact file path and the range of lines and columns where the symbol is defined are packaged into a JSON response. If no definition is found, a response indicating a null result is returned. This response allows the LSP client (the user's text editor or IDE) to navigate directly to the symbol's definition or handle the case where no definition exists.

4.3 File Synchronization & Diagnostics

File synchronization is a key feature of the LSP. Three types of requests are sent by VS Code to notify document opening, changing and saving actions performed by the user. A single file parsing is performed each time a file is opened or saved. We add an event listener to catch any error or warning raised by the Frama-C parser and we store them inside a JSON block. All diagnostics are published to the editor and displayed to the user in the form of squiggles and listed in the 'Problems' subsection of the VS Code terminal/console (?).

There are two types of diagnostics: warnings and errors. Warnings are raised in case an ACSL annotation is not formed properly or when an unknown function is called. Errors are raised when there is a syntax error in the C code preventing the parser to finish (which are fatal errors). In the latter case, normally, any Frama-C session stops, but we ensure that diagnostics are sent either in case the parsing succeeds or fails. So we send them just before the exception aborts fatally. In case the parsing succeeds without any messages, the diagnostics are cleared in the editor.

The listener is a Frama-C function that reports any event emitted by Frama-C while parsing. From those events we can extract their source, their message and their kind (whether it is an error, a warning a feedback, etc.). By manipulating these data, we organize them and adapt them to the LSP standard.

This feature is only implemented for single file parsing and diagnostics are computed only when the file is saved. No diagnostics are published for multiple file parsing, which means we do not know if those files are in state of error. So the user has to make sure the source files they provided have no errors.

4.4 File parsing

In order to use the Frama-C module's functions, the plug-in converts the source code files into their C Intermediate Language (CIL) representation, which provides a detailed hierarchical structure of the source code. It captures the syntactic elements and relationships within the code, for deeper analysis and processing. This representation allows the server to efficiently navigate and manipulate the code, supporting features like 'Go To Definition' and other code navigation tools.

The system is designed to handle the parsing of both single and multiple source code files. Regardless of the number of files being parsed, the resulting Abstract Syntax Trees (ASTs) from each file are merged into a single one. This merging process is crucial as it simplifies the analysis and manipulation of the code, providing a unified structure that the server can work with more effectively.

When dealing with large codebases, users often face the challenge of working with a vast number of source files. Parsing all these files can be a time-consuming task. To address this issue, the system offers a Visual Studio Code setting that allows users to specify which source files need to be parsed. By focusing the parsing process on only the relevant files, this setting helps optimizing the time spent on parsing, making the workflow more efficient.

Users must ensure that all necessary source files are included in the parsing process. If a user fails to specify a source file that is needed for certain features, such as Go To Definition, they may not function correctly. It is therefore the responsibility of the user to provide all the relevant files.

5 LSP use cases with VS Code

In this enclosed figure is a listing of all LSP features, if they are handled by VS Code, and their implementation status on the server side.

tableau des capacités de vs code

6 Illustration of the implementation

commenter les screenshots de chaque feature

7 Future work

tableau des features à implémenter